

Best Practices for Documentation

FFA_regional-skew Project

Contents

1	Overview	2
2	Workflow Status Level	2
2.1	Table 1: Common Status Levels for Data Workflow	2
3	Working with Git	3
3.1	Commit Message Format	3
3.2	Table 3 Common Prefixes (<type>)	3
3.3	Sample Commit Messages	3
3.4	Tips for Good Commit Messages	3
3.5	Git Push Summary Table:	4
4	GitHub Issue Comments:	4
5	Semantic Versioning (SemVer)	4
5.1	SemVer Update Table	4
5.2	Practical Examples	4
5.3	When to Increment Which?	5
5.4	Suggested Tag Format	5
5.5	CHANGELOG Format (Keep a Changelog)	5
6	When to Create a GitHub Issue Comment or Changelog Entry	6
7	Release Checklist (Markdown or Issue Template)	6
8	Don't Repeat Yourself (DRY) Principles for Docs	7
8.1	DRY Benefits	7
8.2	Rule of Thumb: "Single Source of Truth"	7
8.3	Should I use a .csv or an .xlsx?	7
8.4	Should I use a .csv or a .txt?	8
8.5	Rule of Thumb	9
9	Definitions	9
9.1	Data Dictionary vs. Attribute Data	9

1 Overview

This guide describes best practices for documenting changes in hydrologic modeling and data science projects. The guide describes:

- Data workflow status,
- Working with Git,
- Semantic versioning,
- Changelog format,
- DRY principles, and
- Definitions for reference materials

2 Workflow Status Level

2.1 Table 1: Common Status Levels for Data Workflow

Status Code	Label	Description
00	Not Started	Task has not been initiated.
01	Queued for Download	Identified for download; pending execution.
02	Downloaded	Data successfully downloaded. No prep has started yet.
03	Reprojected	Spatial data reprojected to standard CRS (e.g., NAD83).
04	Cleaned / Filtered	Unwanted data removed; NA handling, format consistency applied.
05	Verified Raw Files	File format, integrity, and metadata validated.
06	Metadata Documented	Dataset and processing steps documented in metadata or data dictionary.
07	Feature Extraction	Covariates or metrics calculated (e.g., slope, mean temp).
08	Joined to Site Data	Covariates or values joined with site records (e.g., gage locations).
09	Finalized & Versioned	Clean, final dataset stored in data/processed/; versioned if needed.
10	Used in Modeling	Dataset actively used in modeling or downstream analysis.

2.1.1 Table 2: In-Progress Modifiers

Modifier	Status Indicator
*	partially done
~	needs review
!	blocker or known issue
	fully validated

- **Note:** Control + Command + Spacebar opens the Emoji & Symbols viewer

3 Working with Git

3.1 Commit Message Format

`<type>: <short summary>`

`<body - optional, wrap at 72 characters>`

3.2 Table 3 Common Prefixes (<type>)

Prefix	When to Use
Feat:	New feature or script
Fix:	Bug fix
Docs:	Documentation (README, Rmd logs, comments)
Refactor:	Code reorganization or cleanup (no behavior change)
Test:	Adding or updating tests
Data:	Adding/removing/modifying data files
Chore:	Routine tasks (renaming, formatting, .gitignore edits)
Build:	Dependency or environment updates
Tag:	Milestone tagging, version bump
Style:	Visual-only changes (e.g., spacing, indentation, cleanup)

3.3 Sample Commit Messages

Feat: add macrozone covariate extraction from Level II shapefile

This adds a new script (03c_macrozone_covariates_l2.R) to assign prairie macroregions based on NA Ecoregion Level II polygons. Output is written to data/processed/spatial/.

Docs: move macrozone covariates Rmd log to reports/

Moved 03c Macrozone Covariates from Level II Ecoregions.Rmd from to_check/ into reports/ to reflect finalized covariate pipeline documentation. Part of Milestone 00 refactor.

Chore: update .gitignore with large file exclusions for GitHub limits

3.4 Tips for Good Commit Messages

- Use the imperative mood: “Add”, not “Added” or “Adds”
- Keep subject lines 50 characters
- Use the body to explain **why** and optionally **how**
- Group commits by milestone or logical step

3.4.1 Practical Suggestion

For minor solo edits, let your commit messages carry the weight. Then, before a release, summarize them into the changelog.

3.5 Git Push Summary Table:

Command	When to Use
<code>git push</code>	On an active branch after upstream is set
<code>git push origin main</code>	First-time push, or when using multiple remotes/branches
<code>git push -u origin main</code>	First push of a new branch to track remote automatically

4 GitHub Issue Comments:

- Use when the change is relevant to tracking progress or collaboration.
- Ideal if you're closing or referencing an issue: `git commit -m "Fix: update structure (#42)"`

5 Semantic Versioning (SemVer)

Semantic versioning (SemVer) provides:

- Clear communication of changes
- Enables automation of version checks
- Plays well with Git tags, changelogs, and CI/CD tools
- Supports dependency tracking (e.g., for R packages, Python modules)

5.1 SemVer Update Table

Level	When to Update	Example
MAJOR	Breaking changes invalidate prev results or code	v2.0.0
MINOR	New features or modules added, no breakage	v2.1.0
PATCH	Bug fixes, metadata tweaks, small improvements	v2.1.1

5.2 Practical Examples

Action	Version Bump
Adding a new download script (e.g., MODIS NDVI)	MINOR
Refactoring file structure without breaking existing code	MINOR
Fixing a broken CRS or path bug	PATCH
Renaming key variables that break older models/scripts	MAJOR
Rewriting the modeling workflow or changing the data join strategy	MAJOR

5.3 When to Increment Which?

Situation	Version	Example
You break the API or change file formats	v2.0.0	
You add a new data field or function (compatible)	v1.1.0	
You fix a bug or typo in documentation/code	v1.0.1	

5.4 Suggested Tag Format

Use lowercase v prefix for GitHub compatibility:

```
git tag -a v0.3-refactor -m "Refactor project structure and file locations"
git push origin v0.3-refactor
```

5.5 CHANGELOG Format (Keep a Changelog)

- Use a CHANGELOG.md file at the root of your project.
- Group minor edits into a single entry under a version:

Recommended Format:

Changelog

All notable changes to this project will be documented in this file.

The format is based on [Keep a Changelog](https://keepachangelog.com/) and this project adheres to [Semantic Versioning](https://semver.org/).

[Unreleased]

Added

- Initial support for batch PRISM climate data download
- Created `10a\_exploratory\_modeling\_check\_missing.R` for handling NA values

Changed

- Improved slope outlier detection method in modeling script

Fixed

- Bug in elevation raster alignment

[v1.2.0] - 2025-05-08

Added

- Integrated GAM modeling with `mgcv` package
- Exported residual plots for model diagnostics

Changed

```

- Updated `README.Rmd` with covariate metadata overview
- Harmonized variable names for terrain and climate covariates

### Deprecated
- Old correlation heatmap script (`10b_corr_old.R`)

---

## [v1.1.0] - 2025-04-15

### Added
- Initial elastic net model with tidymodels
- Ecoregion zonal majority mask added

---

## [v1.0.0] - 2025-04-01

### Added
- Project structure and milestone folders
- USGS site metadata download script
- Data dictionary template and preamble

```

6 When to Create a GitHub Issue Comment or Changelog Entry

Change Type	GitHub Issue Comment	Changelog Entry
Major feature or milestone	Yes	Yes
Fix for a reported bug	Yes	Yes
Refactor that affects structure	Optional	Yes
Minor formatting or typo fix	No	No
Cleanup (e.g., deleting old files)	No	Optional
Data or output regeneration only	No	Optional

= Do it if you're tracking tightly or have collaborators who care about visibility

7 Release Checklist (Markdown or Issue Template)

Use this to ensure every release is polished and complete.

Example: RELEASE_CHECKLIST.md

Release Checklist for Version X.Y.Z

Versioning

- [] Bump version (MAJOR / MINOR / PATCH) in tag and `README` if needed
- [] Create annotated Git tag: `git tag -a vX.Y.Z -m "Release vX.Y.Z"`

Documentation

- [] Update `CHANGELOG.md`
- [] Update `README.md` or `README.Rmd` (if applicable)

```

- [ ] Confirm that all scripts are commented and up-to-date
- [ ] If R package: update `DESCRIPTION` and `NEWS.md`

## Code and Data
- [ ] Validate all outputs in `data/` and `outputs/`
- [ ] Confirm that intermediate files are not committed unless intentional
- [ ] Check that all scripts run without error from a clean clone

## Testing
- [ ] Run model scripts and validate final output summaries
- [ ] Visually inspect maps and plots for clarity
- [ ] Check regression residuals and other diagnostics

## Deployment
- [ ] Push all commits and tag to GitHub: `git push && git push origin --tags`
- [ ] Attach `CHANGELOG` section to GitHub release page
- [ ] (Optional) Notify collaborators, update shared folder, archive versioned ZIP

## Optional Extras
- [ ] Archive important rasters or scripts to `release/vX.Y.Z/`
- [ ] Create reproducibility snapshot (e.g., via `renv::snapshot()` in R)

```

8 Don't Repeat Yourself (DRY) Principles for Docs

8.1 DRY Benefits

- Avoid double edits — update once in .csv, it's reflected in .Rmd
- Facilitates plotting, filtering, joining in modeling scripts
- Keeps narrative lighter and cleaner

8.2 Rule of Thumb: “Single Source of Truth”

- Keep the data in the spreadsheet (.csv.xlsx)
- Reference and summarize it in the RMarkdown file (.Rmd), with clear sourcing.

8.2.1 Suggested Division of Labor

Task	.csv or .xlsx	.Rmd
Store raw metadata	Yes	No
Include scale, units, description, notes	Yes	selectively
Discuss rationale, sources, assumptions	No	Yes
List variables by modeling role (e.g., climate, terrain)	Yes (structured)	Yes (narrative/justified)
Drive reproducibility	Yes	Yes
Long-term editable source	Yes	Only generated from spreadsheet

8.3 Should I use a .csv or an .xlsx?

- Use CSV as your primary format when:
- You're working with R scripts, RMarkdown, or version control (Git)

- You want portable, long-term, cross-platform storage
- You need clean diffs in Git (XLSX is binary, CSV is plaintext)
- You're scripting data loads (`readr::read_csv()`, `fread()`, `data.table`, etc.)
- Avoid XLSX when:
 - You don't need multiple sheets, formatting, or formulas
 - You're trying to maintain reproducibility and auditability
 - You're storing version-controlled data (XLSX is not Git-friendly)

8.4 Should I use a .csv or a .txt?

8.4.1 Use .csv when:

- You're working with tabular data (rows and columns)
- You need structured input for R, Python, Excel, GIS, or databases
- You want to join/merge datasets programmatically
- You're using comma-separated values (or semicolons in some locales)
- You need to easily visualize or edit the data in spreadsheet tools

8.4.2 Examples:

- * Covariate metadata for a model: `regional_skew_covariates_metadata_v01.csv`
- * Climate class lookup table: `koppen-geiger_class_legend.csv`
- * Peak flow summary: `data_pk_summary.csv`

8.4.3 Use .txt when:

- You're storing plain text, such as:
 - Notes, READMEs, checklists, or citations
 - Legends or lookups not meant for tabular use
 - License or metadata files with flexible formatting
 - The content doesn't fit neatly into rows and columns
 - You want the file to be human-readable, regardless of software
 - You're sharing instructions, documentation, or a log

8.4.4 Examples:

- * `README.txt`, `RELEASE_NOTES.txt`
- * `license.txt`
- * `koppen-geiger_legend.txt` (if it's just reference info)

8.5 Rule of Thumb

- If it's structured data, use .csv.
- If it's unstructured or semi-structured text, use .txt.

9 Definitions

9.1 Data Dictionary vs. Attribute Data

Attribute data refers to the actual values associated with each record or feature in a dataset. In a spatial dataset, these are typically the values in the attribute table (e.g., names, measurements, categories).

A *Data Dictionary* A data dictionary is metadata — it describes the structure, meaning, and units of the fields in your dataset. It helps humans understand what each field represents. It can be saved as:

- A separate file (e.g., Excel, CSV, JSON, R script)
- As a documentation section in an RMarkdown or README
- Embedded in metadata for shapefiles (e.g., .xml files) or GeoPackages

9.1.1 Data Dictionary Example

Field Name	Description	Unit	Data Type
STATE_NAME	Name of the U.S. state	NA	Text
POP_2020	Population in 2020	Persons	Integer
AREA_SQKM	Area of the state	sqkm	Float